UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Software Architectures

Practice 8

Zoltán Richárd JÁNKI, PhD

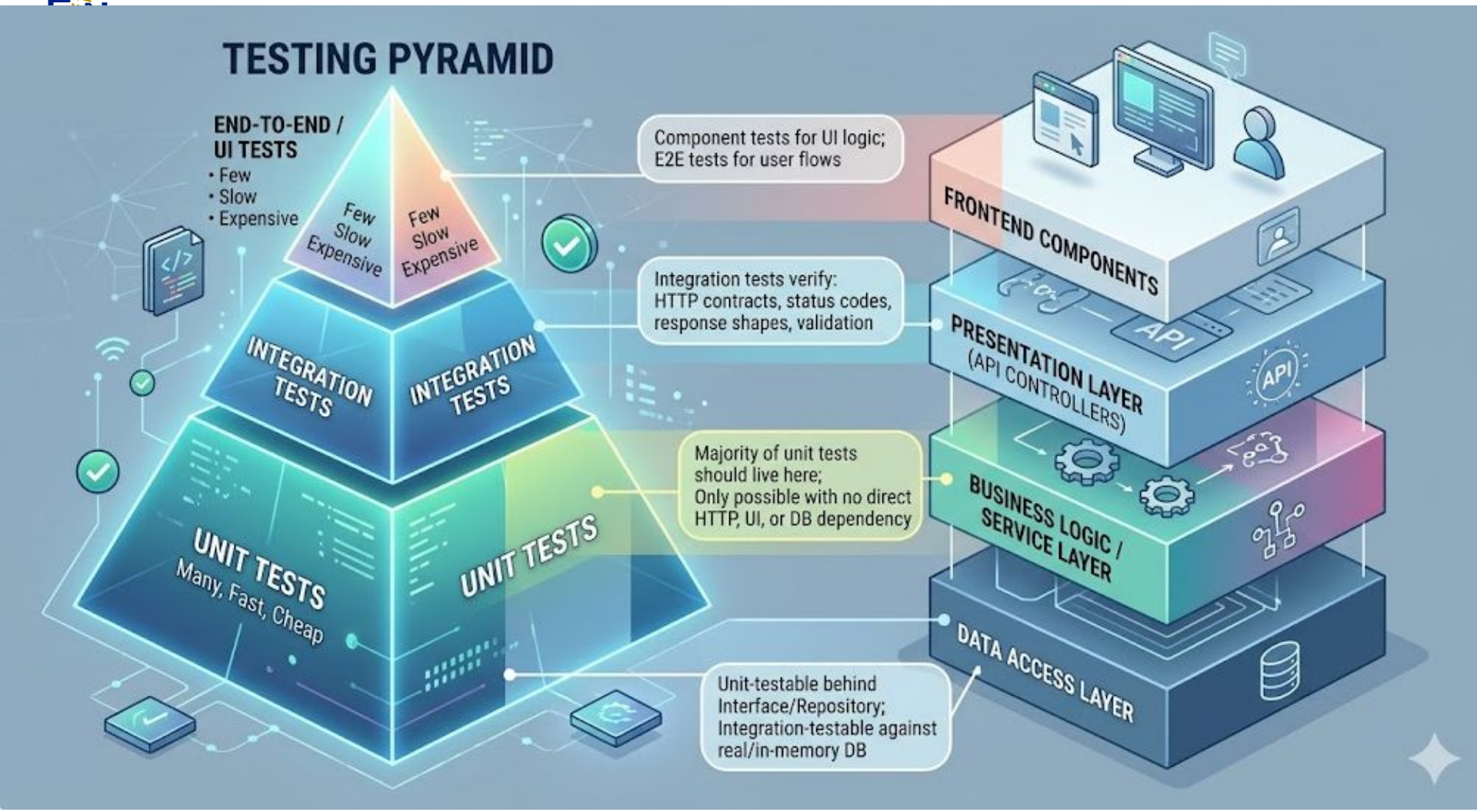
Testability

Testability

- Testability = the degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met

- Observability — can we see what's happening inside the system? Can we inspect the output, the state, the side effects?
- Controllability — can we put the system into a specific state before running a test? Can we replace dependencies with controlled alternatives?

Testing Pyramid



Popular test frameworks

- Backend (Node.js)
  - Jest
  - Mocha+Chai+Sinon
  - Vitest
- Frontend (Angular)
  - Jasmine+Karma
  - Jest
- E2E
  - Cypress
  - Playwright

Dependency Inversion

- Never create or import directly a dependency
  - It is hard to test: slow, fragile, requires setup
  - E.g.: a service imports a database module and calls its function → you need a real running database
- Instead use a “repository” that is a parameter
  - It (data repository) can be mocked
  - E.g.: in production it uses the real repository → for testing you can just pass mock data (skipping the database)

Architecture review checklist for testability

- **Can each layer be instantiated independently?** *If your service layer requires the entire Express app to start, that's a problem.*
- **Are external dependencies (database, APIs, file system) behind abstractions?** *If you're calling prisma.user.findMany() directly inside your route handler, you can't unit test that handler without Prisma and a database.*
- **Does business logic live in its own layer, free from HTTP and UI concerns?** *If your validation logic or calculation logic is in a controller or a React component, it can't be unit tested cleanly.*
- **Are there pure functions where possible?** *Pure functions (same input → same output, no side effects) are the easiest thing to test. Identify logic that could be extracted into pure functions.*
- **Is the application state manageable in tests?** *Can you set up a known starting state, run an action, and verify the result? If your app relies on global mutable state, this becomes very difficult.*
- **Are API contracts well-defined?** *If you have an OpenAPI spec, you can auto-generate contract tests that verify your API matches the spec.*

Common Anti-Patterns

- **"Fat Controllers" / "Fat Components":** *The route handler or React component does everything: validation, business logic, database calls, response formatting. You can't test any one concern without triggering all of them.*
- **Hard-coded dependencies:** *Directly importing and using concrete implementations everywhere. No way to substitute a test double.*
- **Business logic in the database layer:** *Complex SQL queries or stored procedures that contain business rules. These can only be tested with a real database, making tests slow and brittle.*
- **Shared mutable global state:** *Singletons or global variables that carry state between operations. Tests affect each other, leading to flaky, order-dependent test suites.*
- **Tight coupling to external services:** *Calling a third-party API directly from business logic. Tests require the external service to be available, or they fail.*